



Branch of Study: Computer Science

Course: IT Project C1

Student Name: Otu-ekong Joseph Effiong

Student ID: 42468

Course Type: Full-time course

Institution Name: University of Economics and Human Sciences in Warsaw
Poland, Warsaw, Ul. Okopowa 59

Documentation for the diploma project prepared under the supervision of **Marcin Kacprowicz**

Academic Year: 2026

SkyLine: A Visa-Aware Low-Cost Flight Planning Web Application for Travelers with Low Passport Mobility

1. Basic Information

1.1. Project Name

SkyLine: a visa-aware low-cost flight planning web application for travelers with low passport mobility.

1.2. Project Goal

The goal of the project is to create a modern web application that helps foreigners and travelers with low passport strength find affordable, realistic, and legally safer flight routes. The application combines flight search, passport and visa eligibility checks, airport transit analysis, and route visualization to recommend travel plans that fit the user's nationality, existing visas, residence status, travel dates, and budget.

Unlike a standard flight search engine that only sorts results by price or duration, SkyLine evaluates whether a traveler is likely to be allowed to enter or transit through the countries included in the route. The system is designed to reduce the risk of choosing a cheap route that requires a transit visa, creates a self-transfer problem, or sends the traveler through an airport that is unsuitable for their passport situation.

1.3. Brief Description of the Project

SkyLine is a full-stack web application for visa-aware flight planning. A user creates a traveler profile containing passport country, nationality, residence country, passport expiry date, and existing visas or residence permits. The user then enters the origin, destination, preferred dates, and budget. The system searches for possible flight routes, checks destination and transit rules, scores each route, and presents the safest low-cost options with a clear explanation.

The application includes an airport search module, a route recommendation engine, a visa and transit rule engine, a flight path map, saved traveler profiles, saved searches, booking records, and payment simulation. The project is especially useful for users whose travel options are limited by visa requirements and passport mobility. The main output is not only a list of flights, but a complete recommended travel plan showing price, duration, layovers, transit countries, visa warnings, and the reason why a route is recommended or rejected.

2. Competitor Product Analysis

2.1. Google Flights

Google Flights is very effective for comparing prices, dates, airlines, and route duration. However, it does not deeply personalize recommendations based on the traveler's passport strength, current residence status, or existing visas. It may show cheap routes that are legally difficult or impossible for some travelers to use.

2.2. Skyscanner and Kiwi.com

Skyscanner and Kiwi.com are strong for low-cost route discovery and flexible date search. Kiwi.com is especially useful for creative routing and virtual interlining. However, these tools still focus mainly on price and availability. They do not act as a passport-aware travel planning assistant and may require the user to manually understand transit visa risk.

2.3. Sherpa and IATA Timatic

Sherpa and IATA Timatic focus on travel restrictions, visa documentation, and entry requirements. They are useful for official travel eligibility checks, but they are not complete flight planning interfaces. They do not usually optimize a route based on price, layover risk, and passport mobility together in one simple workflow.

2.4. FlightAware and OpenSky

FlightAware provides strong operational flight status information, while OpenSky provides aircraft position data. These tools are useful for tracking and visualization, but they are not booking systems and do not solve visa-aware route planning.

Compared to these products, SkyLine combines the main needs of a specific user group: low-cost flight planning, visa and transit awareness, passport-strength consideration, and route visualization. The application focuses on helping the user choose a route that is affordable and legally realistic, not simply the cheapest result.

3. List of Technologies Used

- React, Vite, JavaScript, Tailwind CSS, shadcn/ui components, and Lucide icons for the frontend.
- Netlify for hosting, HTTPS, deployment, environment variables, and serverless backend functions.
- Supabase for authentication, PostgreSQL database storage, saved traveler profiles, saved searches, and booking records.
- Duffel API for flight search, pricing, passenger data, and order creation.
- Kiwi.com Tequila API as an optional provider for low-cost and flexible route discovery.
- FlightAware AeroAPI for flight status, delays, gates, terminals, and operational tracking.
- OpenSky Network for free aircraft position data and map-based tracking.
- OurAirports dataset for airport codes, airport search, city matching, and coordinates.
- Custom visa and transit rule tables in Supabase for the diploma implementation.
- IATA Timatic or Sherpa as possible future professional sources for travel-documentation rules.
- Stripe test mode for payment simulation before booking confirmation.

4. Technological Stack and Justification

React and Vite were selected for the frontend because they provide a fast development environment and a modern structure suitable for a responsive travel planning interface. Tailwind CSS and shadcn/ui allow the interface to be built quickly with consistent components such as forms, route cards, alerts, dialogs, filters, and status badges.

Supabase was selected as the database and authentication platform because the project requires relational data: users, traveler profiles, visas, passport information, saved searches, route recommendations, payments, and bookings. PostgreSQL is a strong fit because these records have clear relationships. Supabase Auth also reduces the amount of custom security code needed for user accounts.

Duffel is the recommended flight booking provider because it offers a modern REST API, sandbox environment, and a clear search, price, and book workflow. For cheaper and more flexible route discovery, Kiwi.com Tequila can be added as an optional search provider. FlightAware AeroAPI is suitable for operational flight status, while OpenSky is suitable for free aircraft position visualization. OurAirports is used locally to avoid unnecessary paid calls for airport lookup and to provide route coordinates for the flight path map.

Netlify is recommended for deployment because the frontend can be built with `npm run build` and published from the `dist` directory. Netlify Functions can hold API keys safely and call external services without exposing secrets in the browser. This is important because keys for Duffel, FlightAware, Stripe, and Supabase service roles must remain private.

5. Key Issues Related to the Implementation of the Project

5.1. Visa-Aware Route Filtering

The most important implementation issue is that the system must not rank flights only by price. For the target users, the cheapest route may be risky if it includes a layover country that requires a transit visa or if it involves self-transfer through immigration. The application therefore needs a rule engine that checks each route against the traveler profile. The engine must consider passport country, destination country, transit countries, existing visas, residence permits, passport expiry, and whether airport transit is allowed.

For the diploma version, the visa and transit rules can be implemented as database tables maintained by the application administrator. Each rule describes whether a traveler from one country can enter or transit through another country, whether an eVisa is available, whether visa on arrival is possible, and whether exceptions apply. In a production version, this data should be replaced or validated by professional services such as IATA Timatic or Sherpa.

5.2. Route Scoring and Recommendation Logic

SkyLine should score each route using several factors. Visa safety has the highest weight because a legal or transit issue can make an otherwise cheap route unusable. Price is also important because the app targets users who need low-cost options. Duration, number of layovers, airport reliability, self-transfer risk, and baggage risk are also included.

A practical scoring model is:

- Visa safety: 45%
- Price: 30%
- Duration: 15%
- Comfort and layovers: 10%

Routes with serious visa conflicts should be rejected or marked as “avoid”, even if they are cheaper. Routes with mild uncertainty should display warnings such as “transit visa may be required”, “check airline baggage transfer”, or “passport expiry risk”.

5.3. Secure API Integration

Flight booking APIs and payment APIs require private keys. These keys must never be stored in Vite frontend variables because browser code can be inspected by users. The project therefore uses a backend proxy through Netlify Functions or a Node.js/Express server. The frontend calls endpoints such as `/api/flights/search`, `/api/flights/price`, `/api/flights/book`, `/api/visa/check`, and `/api/payments/create-intent`. The backend then calls Duffel, FlightAware, Stripe, and Supabase securely.

5.4. Airport Search and Flight Path Visualization

The application needs reliable airport data for search and maps. The OurAirports dataset can be imported into Supabase with airport names, IATA codes, ICAO codes, city, country, latitude, and longi-

tude. When a route is displayed, the app can draw the path from the origin airport to each layover and then to the destination. Each stop can show visa or transit warnings directly on the map or route card.

5.5. Data Privacy and Sensitive Traveler Information

The app stores sensitive profile information such as passport country, nationality, visas, and travel history. The implementation must minimize stored data and avoid saving unnecessary full passport details unless required. Supabase row-level security should be enabled so users can only access their own profiles, saved searches, and bookings. Payment details should not be stored directly in the application database; Stripe handles card data.

5.6. Handling Incomplete or Uncertain Travel Rules

Visa rules change frequently and can depend on many details. A diploma project can demonstrate the decision process with a controlled dataset, but the interface must clearly show that rules should be verified before travel. The system should mark uncertain cases and explain why a warning is shown. This improves trust and makes the recommendation logic transparent.

6. Suggested Screenshots and Visualizations

The final project documentation should include the following screenshots:

- Home/search screen with origin, destination, dates, budget, nationality, passport country, and existing visa fields.
- Traveler profile screen showing saved passport country, residence country, and visas.
- Route results screen showing recommended, warning, and avoid route cards.
- Flight path map showing origin, layovers, destination, and transit visa warnings.
- Booking/checkout screen using Stripe test mode and saving the booking to Supabase.
- Admin/rules screen for managing visa and transit rules in the diploma demo.

7. Conclusions and Development Prospects

SkyLine addresses a practical problem that is not fully solved by ordinary flight search engines. Many travelers cannot choose flights based only on price because their passport strength, residence status, and visas affect which destinations and transit routes are possible. The project demonstrates how flight APIs, airport data, user profiles, and visa/transit rules can be combined into a recommendation system that provides safer and more useful travel planning.

The first version can use a controlled visa rule dataset and API sandbox environments. Future development could include integration with IATA Timatic or Sherpa for professional travel documentation checks, wider booking inventory through Sabre or Travelport, automatic passport document scanning, multilingual support, mobile applications, route risk learning based on user feedback, and stronger admin tools for maintaining visa rules.

8. Bibliography and Sources

- Duffel API Documentation: <https://duffel.com/docs/api/overview/welcome>
- Kiwi.com Tequila API Documentation: https://tequila.kiwi.com/portal/docs/tequila_api
- FlightAware AeroAPI Documentation: <https://www.flightaware.com/aeroapi/portal/documentation>
- OpenSky Network REST API: <https://openskynetwork.github.io/opensky-api/rest.html>
- OurAirports Data: <https://ourairports.com/data/>
- Supabase Documentation: <https://supabase.com/docs>
- Stripe API Documentation: <https://docs.stripe.com/api>
- Netlify Functions Documentation: <https://docs.netlify.com/build/functions/overview/>
- IATA Timatic: <https://www.iata.org/en/services/compliance/timatic/>
- Sherpa: <https://www.joinsherpa.com/>
- Passport Index: <https://www.passportindex.org/>
- Henley Passport Index: <https://www.henleyglobal.com/passport-index>